# APU GUARD

Sistema de Detección de Deforestación — Reserva Nacional Tambopata

## Alcance Técnico Implementado

Backend · Frontend

| | |
|---|---|
| **Proyecto** | APU GUARD — TP1 Ingeniería de Software |
| **Repositorio** | apuguard_deploy_new/apuguard_deploy |
| **Stack** | Django 4.2 · Angular 18 · PostgreSQL/PostGIS · AWS |
| **Fecha** | Junio 2026 |

## 1. Introducción

APU GUARD es una plataforma web de monitoreo y detección temprana de deforestación en la Reserva Nacional Tambopata, Madre de Dios, Perú. El sistema integra imágenes satelitales Sentinel-2, un modelo de segmentación semántica HRNet desplegado en AWS SageMaker, y una interfaz web para que analistas ambientales gestionen alertas, coordinen técnicos de campo y generen reportes mensuales.

**Alcance de este documento:** Describe exclusivamente las funcionalidades que fueron efectivamente implementadas y probadas en el entorno de desarrollo. No incluye componentes planificados pero no ejecutados.

## 2. Arquitectura General

La aplicación sigue una arquitectura de tres capas desacopladas, orquestada con Docker Compose para desarrollo local:

| Capa | Tecnología | Puerto | Función principal |
|---|---|---|---|
| Base de datos | PostgreSQL 16 + PostGIS | 5432 | Almacenamiento relacional y geoespacial |
| Message broker | Redis 7 | 6379 | Cola de tareas Celery |
| Backend API | Django 4.2 + DRF | 8000 | REST API, lógica de negocio, pipeline ML |
| Worker | Celery Worker | — | Procesamiento asíncrono SageMaker |
| Scheduler | Celery Beat | — | Tareas periódicas quincenales |
| Frontend | Angular 18 | 4200 | Interfaz web SPA |

# 3. Backend

## 3.1 Stack tecnológico

| Django 4.2 | Django REST Framework | SimpleJWT | GeoDjango / PostGIS | Celery 5 | boto3 | Twilio | reportlab |
|---|---|---|---|---|---|---|---|

## 3.2 Modelos de datos implementados

El esquema de base de datos está compuesto por 9 modelos Django organizados en cinco dominios funcionales:

| Dominio | Modelo Django | Descripción |
|---|---|---|
| Usuarios y roles | auth.User | Usuario nativo Django. is_staff=True → Administrador |
| | PerfilAnalista | OneToOne con User. M2M con SectorReserva para filtrado por zona |
| Territorio | SectorReserva | Polígono PostGIS de cada sector de la reserva (activo/inactivo) |
| Contactos | TecnicoCampo | Contacto SMS de campo. NO es usuario de login. M2M con sectores |
| | CapturaSatelital | Imagen Sentinel-2 descargada. Registra ruta S3 y cobertura de nubes |
| | ParcheProcesamiento | Parche 256×256 px enviado a SageMaker. Registra estado y request_id |
| Alertas | CasoAuditoria | Resultado de detección. Geometría PostGIS, score, estado ecológico, ciclos limpios |
| Comunicaciones | RegistroSMS | Historial de cada mensaje SMS enviado vía Twilio por caso |
| Reportería | DocumentoReporte | Reporte mensual CSV almacenado en S3 con métricas agregadas |

## 3.3 API REST — Endpoints implementados

Todos los endpoints requieren autenticación JWT Bearer Token, salvo /api/auth/token/.

| Método | Endpoint | Descripción | Acceso |
|---|---|---|---|
| POST | `/api/auth/token/` | Login — obtener par de tokens JWT (access + refresh) | Público |
| POST | `/api/auth/token/refresh/` | Renovar access token usando refresh token | Público |
| GET | `/api/usuarios/me/` | Perfil del usuario autenticado | Todos |
| PATCH | `/api/usuarios/me/` | Actualizar nombre, email del propio perfil | Todos |
| GET | `/api/usuarios/` | Listar todos los usuarios del sistema | Admin |
| POST | `/api/usuarios/` | Crear nuevo usuario (Administrador o Analista) | Admin |
| PATCH | `/api/usuarios//` | Editar usuario: rol, estado, sectores asignados | Admin |
| DELETE | `/api/usuarios//` | Soft-delete: desactiva el usuario sin borrarlo | Admin |
| GET | `/api/tecnicos/` | Listar técnicos de campo activos | Admin |
| POST | `/api/tecnicos/` | Crear técnico de campo con sector asignado | Admin |
| PATCH | `/api/tecnicos//` | Editar técnico (nombre, teléfono, sector) | Admin |
| DELETE | `/api/tecnicos//` | Desactivar técnico de campo | Admin |
| GET | `/api/alertas/` | Listar alertas con paginación y filtros por fecha/zona | Admin / Analista |
| GET | `/api/alertas//` | Detalle de alerta con tipo, confianza, estado ecológico, SMS | Admin / Analista |
| PATCH | `/api/alertas//` | Actualizar estado y notas técnicas del caso | Admin / Analista |
| POST | `/api/alertas//notificar/` | Enviar SMS manual a técnicos seleccionados vía Twilio | Admin |

| | | | |
|---|---|---|---|
| GET | `/api/alertas//geojson/` | Exportar geometría del área afectada como GeoJSON Feature | Admin / Analista |
| GET | `/api/alertas//pdf/` | Generar y descargar PDF de reporte del caso (reportlab) | Admin / Analista |
| POST | `/api/alertas//evidencia/` | Generar presigned PUT URL S3 para subir foto de campo | Admin / Analista |
| GET | `/api/sectores/` | GeoJSON FeatureCollection de sectores (filtrado por rol) | Admin / Analista |
| GET | `/api/sectores/simple/` | Lista plana de sectores para poblar selectores | Admin / Analista |
| GET | `/api/casos/` | GeoJSON de casos de auditoría para el mapa (filtros disponibles) | Admin / Analista |
| GET | `/api/auditoria//` | Detalle completo del caso + URLs presignadas S3 para visualización | Admin / Analista |
| PATCH | `/api/auditoria//` | Emitir veredicto humano sobre el caso | Admin |
| GET | `/api/dashboard/stats/` | KPIs del panel: alertas activas, área afectada, pipeline | Admin / Analista |
| POST | `/api/pipeline/trigger/` | Disparar manualmente el pipeline de detección quincenal | Admin |
| GET | `/api/reportes/` | Listar reportes mensuales con presigned URL S3 de descarga | Admin |

### 3.4 Sistema de roles y permisos

La aplicación implementa tres niveles de acceso diferenciados a través de clases de permiso personalizadas en Django REST Framework:

| Rol | Mecanismo | Acceso |
|---|---|---|
| Administrador | is_staff = True en auth.User | Acceso total: usuarios, técnicos, pipeline, reportes, SMS manual |
| Analista | PerfilAnalista + sectores M2M | Solo ve alertas y sectores de sus zonas asignadas. Sin gestión de usuarios |
| Técnico de Campo | TecnicoCampo (no usuario de login) | Solo recibe SMS automáticos. No tiene acceso a la interfaz web |

### 3.5 Pipeline de detección (Celery)

El procesamiento ML es completamente asíncrono. Celery Beat programa la tarea quincenal y Celery Worker la ejecuta en segundo plano:

| Paso | Tarea Celery | Descripción |
|---|---|---|
| 1 | procesar_quincena_tambopata | Descarga imagen Sentinel-2 de GEE, divide en parches 256×256, sube tensores .npy a S3 (sagem |
| 2 | procesar_resultado_sagemaker | Lee resultados .npz de S3 (sagemaker-outputs/), aplica umbral de entropía, crea CasoAuditoria pa |
| 3 | generar_reporte_mensual | Agrega métricas del mes (casos validados, recuperados, SMS enviados) y sube CSV a S3 (apugu |
| 4 | limpiar_archivos_temporales_s3 | Elimina visualizaciones PNG de S3 (sagemaker-visualizations/) con más de 30 días de antigüedad |

### 3.6 Integraciones externas

| Servicio | Uso en la aplicación |
|---|---|
| AWS S3 | Almacenamiento de rasters Sentinel-2, tensores, resultados SageMaker, visualizaciones PNG, evidencias fotográfica |
| AWS SageMaker | Inferencia asíncrona del modelo HRNet. Se usa AsyncInferenceConfig con OutputLocation en sagemaker-outputs/ |
| Google Earth Engine | Descarga de imágenes Sentinel-2 L2A (10m) para la ventana quincenal usando la API de GEE con autenticación de |
| Twilio | Envío de SMS de alerta a técnicos de campo. Registra cada envío en RegistroSMS con estado ENVIADO/FALLIDO |

# 4. Frontend

## 4.1 Stack tecnológico

| Angular 18 | Standalone Components | Signals | Tailwind CSS | Leaflet | CommonModule |
|---|---|---|---|---|---|

El frontend es una Single Page Application (SPA) construida con Angular 18 usando el patrón de Standalone Components (sin NgModules). El estado reactivo se gestiona exclusivamente con Signals de Angular, sin librerías externas de gestión de estado. El diseño visual usa Tailwind CSS con clases utilitarias personalizadas definidas en styles.css.

## 4.2 Pantallas implementadas

### Acerca del Proyecto (pública) /acerca

Página informativa accesible sin autenticación. Presenta el contexto del problema de deforestación, la arquitectura general del sistema (Sentinel-2, HRNet, SageMaker, Twilio) y el área de cobertura de la Reserva Tambopata. Incluye el logo de APU GUARD y botón de acceso al sistema.

- Presentación pública del proyecto sin necesidad de login
- Navegación al formulario de inicio de sesión

### Login /login

Formulario de autenticación con panel izquierdo verde de marca y panel derecho con campos de credenciales. Realiza POST a /api/auth/token/ y almacena el JWT en localStorage. Redirige al Dashboard tras autenticación exitosa.

- Autenticación con usuario y contraseña
- Manejo de errores 401 con mensaje de feedback
- Redirección automática post-login al Dashboard

### Dashboard /dashboard

Panel principal con KPIs en tiempo real y mapa interactivo Leaflet. El contenido se adapta al rol: el Administrador ve métricas globales mientras el Analista ve solo sus sectores asignados.

- KPIs: alertas activas, área afectada total (Ha), usuarios activos (Admin), sectores asignados (Analista)
- Estado del sistema y última imagen satelital procesada
- Mapa Leaflet con sectores en GeoJSON coloreados por estado
- Capa de alertas sobre el mapa con popup que muestra ID, zona y área
- Toggle para mostrar/ocultar capa de sectores
- Botón de disparo manual del pipeline de detección (solo Admin)
- Indicador de pasos del pipeline (descarga → filtro nubes → segmentación)

### Gestión de Alertas `/alertas`

Tabla paginada de alertas con filtros y modal de detalle completo. El Analista solo ve alertas de sus sectores asignados (filtrado en backend). El Administrador ve todas las alertas del sistema.

- Tabla con columnas: ID, Fecha, Zona, Coordenadas GPS, Estado, Acciones
- Filtros por rango de fechas y búsqueda por nombre de zona
- Badge de estado coloreado (Pendiente / Validada en campo / Descartada)
- Modal de detalle con: coordenadas, área afectada, tipo de detección, nivel de confianza (barra de progreso), estado ecológico, ciclos quincenales sin actividad
- Historial de notificaciones SMS por caso: técnico, teléfono, fecha y estado del envío
- Upload de evidencia fotográfica de campo directamente a S3 mediante presigned PUT URL
- Edición de notas y observaciones técnicas del caso
- Exportar geometría del área afectada como archivo GeoJSON
- Descargar reporte PDF del caso generado en el backend con reportlab
- Botón Notificar SMS: selección de técnicos y envío manual (solo Admin)
- Toggle de estado: validar o reabrir alertas

### Gestión de Usuarios `/usuarios`

Panel exclusivo para Administradores. Permite crear, editar y desactivar usuarios del sistema con asignación de rol y sectores.

- Tabla de usuarios con nombre, correo, rol, sectores asignados, estado y último acceso
- Filtro por rol (Administrador / Analista) y por estado (Activo / Inactivo)
- Búsqueda por nombre o correo electrónico
- Modal de creación: nombre, correo, usuario, contraseña, rol, sectores (multi-select para Analistas)
- Modal de edición: mismos campos, sin requerir contraseña
- Soft-delete: desactiva el usuario sin eliminarlo de la base de datos
- Fecha de último acceso actualizada automáticamente vía UPDATE_LAST_LOGIN de JWT

### Gestión de Contactos `/contactos`

Administración de técnicos de campo que reciben alertas por SMS. Los técnicos no son usuarios del sistema, son solo contactos con número telefónico.

- Tabla de técnicos con nombre, teléfono, email, sector asignado y estado
- Modal de creación y edición con asignación de sector
- Desactivación de técnicos sin eliminación permanente

### Configuración `/configuracion`

Panel de parámetros globales del sistema accesible solo para Administradores.

- Configuración del umbral de recuperación: número de ciclos quincenales consecutivos sin actividad requeridos para marcar un sector como RECUPERADO

### 4.3 Servicios y arquitectura interna

La lógica de comunicación con el backend está centralizada en servicios Angular inyectables, siguiendo el patrón de separación de responsabilidades:

| Servicio | Responsabilidad |
|---|---|
| AuthService | Login/logout, almacenamiento del JWT, señales reactivas de usuario autenticado y rol (isAdmin). Interceptor HTTP q |
| ApiService | Todos los métodos HTTP hacia la API REST: getAlertas, getAlerta, patchAlerta, notificarSms, solicitarUrlEvidencia, c |

### 4.4 Modelo de datos del frontend

Las interfaces TypeScript en alerta.model.ts y user.model.ts tipan todas las respuestas de la API. Los tipos clave implementados son:

| Interface / Type | Campos principales |
|---|---|
| Alerta | id, codigo, fecha, zona, coordenadas, area_ha, severidad, estado, tipo_caso, score_confianza, estado_ecologico, |
| RegistroSms | id, tecnico_nombre, tecnico_telefono, fecha_envio, estado, error_detalle |
| TecnicoCampo | id, nombre, telefono, email, sector, sectores, activo |
| Sector | id, nombre, codigo, geometry |
| Usuario | id, username, email, nombre_completo, rol, estado, sectores_ids, fecha_ultimo_acceso |
| Reporte | id, anio, mes, casos_validados, casos_recuperados, total_sms_enviados, s3_presigned_url |

## 5. Resumen de lo implementado

| Componente | Cantidad | Detalle |
|---|---|---|
| Endpoints REST | **27** | Cubriendo auth, usuarios, técnicos, alertas, sectores, casos, dashboard, pipeline y reportes |
| Modelos Django | **9** | Con soporte PostGIS para geometrías de sectores y casos de auditoría |
| Migraciones | **4** | Incluyendo migración de PerfilAnalista con relación M2M a sectores |
| Tareas Celery | **4** | Pipeline quincenal, resultados SageMaker, reporte mensual, limpieza S3 |
| Permisos personalizados | **2** | IsAdministrador e IsTecnicoOAdministrador |
| Pantallas Angular | **7** | Acerca, Login, Dashboard, Alertas, Usuarios, Contactos, Configuración |
| Servicios Angular | **2** | AuthService y ApiService con interceptor JWT automático |
| Integraciones externas | **4** | AWS S3, AWS SageMaker, Google Earth Engine, Twilio SMS |
| Generación de PDF | **Sí** | Backend con reportlab: reporte de caso con metadatos, IA, SMS e historial |
| Upload a S3 | **Sí** | Evidencia fotográfica de campo mediante presigned PUT URL directa |
| Exportación GeoJSON | **Sí** | Geometría del área afectada descargable desde el modal de alerta |

**Nota sobre el alcance:** El pipeline de detección ML (Celery + SageMaker) está implementado y es funcional en código, pero su ejecución completa en producción depende de credenciales AWS activas y del modelo HRNet empaquetado en model.tar.gz. La interfaz web, la API REST y la gestión de alertas fueron probadas en entorno local con Docker Compose y datos de prueba reales cargados en PostgreSQL.